# Fourier Neural Operators for Parametric Darcy Flow: What Really Drives Accuracy

Aryamann Srivastava, Varun Sathaye, Atishay Jain, Vedant S. Tiwari | Team PINNacles
Department of Aerospace Engineering, IIT Kanpur | AE646: Scientific Machine Learning for Fluid Mechanics | November 2026

**0.0202** — final FNO test error (MLP: 0.0415)

**2 × / 10 ×** — more accurate / fewer parameters than the MLP

**0.052 → 0.025** — error halved by the training recipe alone

**5–35 ×** — faster than a finite-volume solver (GPU)

## 1 Problem and data

Steady flow in a porous medium: predict the pressure $u$ from the permeability map $\kappa$,

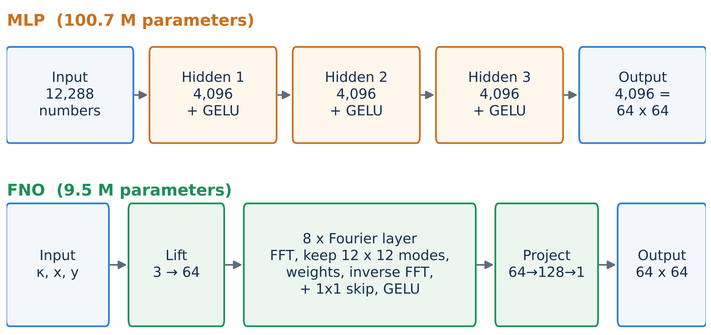
$$-\nabla \cdot (\kappa \nabla u) = 1 \text{ in } (0,1)^2, \qquad u = 0 \text{ on the boundary,}$$

with $\kappa \in \{0.1, 1.0\}$ (thresholded random field). We learn the **operator** $\mathcal{G} : \kappa \mapsto u$, so a new map is answered in one forward pass, without a solve.

- **Data:** PDEBench 2D Darcy ($\beta = 1$), 10,000 real solver outputs at $128^2$, subsampled to $64^2$.
- **Split** (fixed, seed 42): 900 train, 100 validation, 200 test. All choices on validation, test used once.
- **Inputs:** $\kappa$ plus $x, y$ coordinates; standardised with training statistics.

Example predictions (best, typical, worst) are in panel 5.

## 2 Two models, same inputs




- **MLP:** flattens the grid into 12,288 numbers. No notion of neighbouring pixels. 100.7 M parameters.
- **FNO:** each Fourier layer transforms to frequencies, keeps the lowest $12 \times 12$ modes, mixes channels with learned complex weights $R(k)$, transforms back and adds a local path:

$$v_{\ell+1} = \sigma(W v_\ell + \mathcal{F}^{-1}[\, R \cdot \mathcal{F}(v_\ell)\,]).$$

By the convolution theorem this is a global convolution at $O(N \log N)$ cost. 9.5 M parameters.

## 3 Training and metric

Per-sample relative $L_2$ error in physical units, with pixel sums ($j$) taken separately in numerator and denominator; the loss averages it over a batch of $B = 16$ maps:

$$\mathcal{L} = \frac{1}{B} \sum_{i=1}^{B} \frac{\sqrt{\sum_j (\hat{u}_{ij} - u_{ij})^2}}{\sqrt{\sum_j u_{ij}^2}}$$

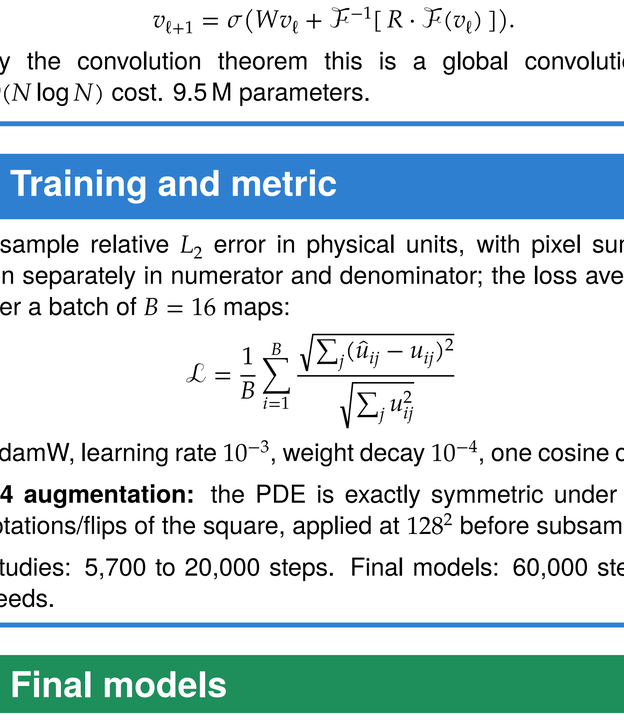
- AdamW, learning rate $10^{-3}$, weight decay $10^{-4}$, one cosine decay.
- **D4 augmentation:** the PDE is exactly symmetric under the 8 rotations/flips of the square, applied at $128^2$ before subsampling.
- Studies: 5,700 to 20,000 steps. Final models: 60,000 steps, 5 seeds.

## 4 Final models

| Model | Params | Mean error | Median |
|---|---|---|---|
| FNO | 9.5 M | **0.0202 ± 0.0004** | 0.0070 |
| MLP | 100.7 M | 0.0415 ± 0.0009 | 0.0272 |

FNO: 12 modes, width 64, 8 layers. MLP: $3 \times 4096$, tuned on validation. 5 seeds, 60,000 steps. The FNO lead grew from 1.6× (Stage 2) to 2× once both models were trained well.

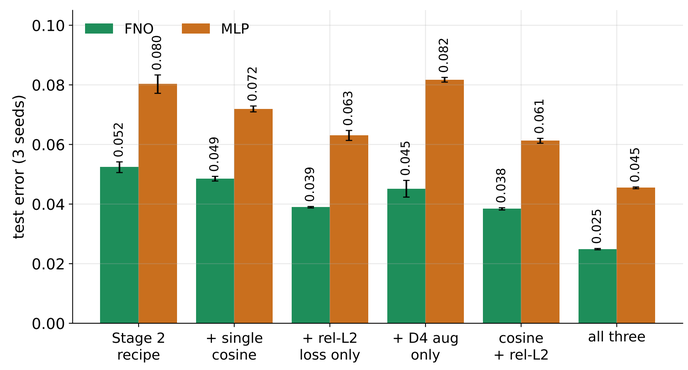
## 5 Example test predictions




Best (0.2 %), typical (0.7 %) and worst (36 %) test maps of the final FNO: permeability, truth, prediction, error. The worst map has uniform $\kappa = 0.1$: right shape, wrong height.

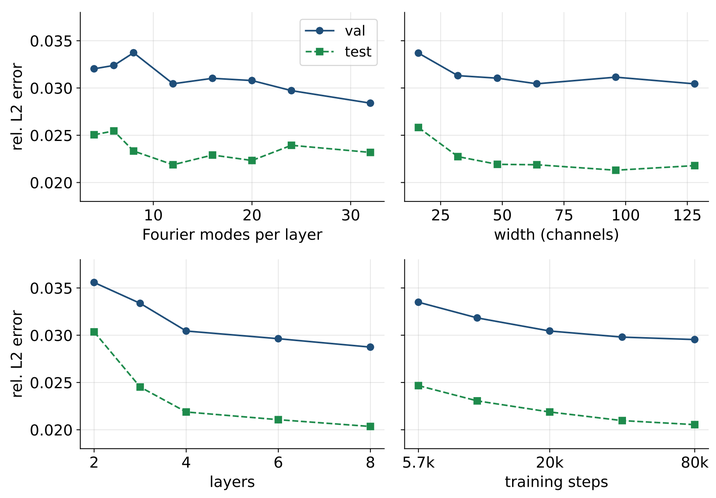
## 6 Finding 1: the training recipe

Stage 2 trained on MSE (the metric is relative), restarted the learning rate every 200 steps and used no augmentation. Fixing all three, same model and budget (3 seeds):



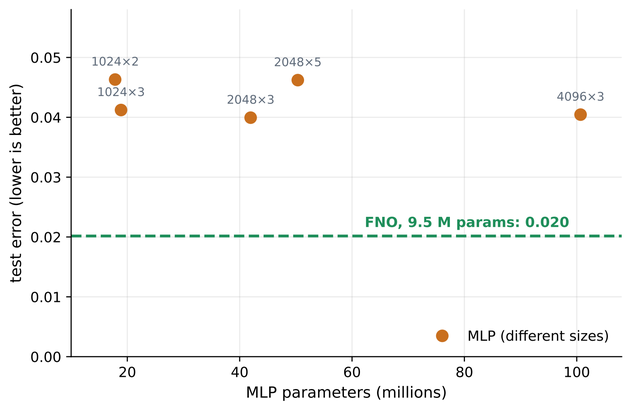

**Same model: FNO 0.052 → 0.025, MLP 0.080 → 0.045**

## 7 Finding 2: architecture matters little



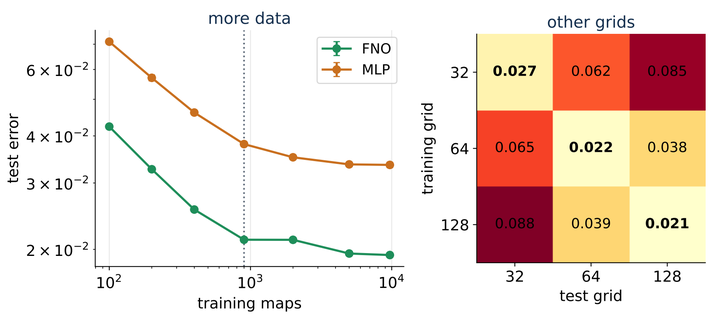

- Modes 4 to 32 and width ≥ 32: nearly flat. Depth 2 → 8 layers: 0.030 → 0.020.
- Batch 8/16/32: 0.0222 / 0.0219 / 0.0221. Learning rate $5 \cdot 10^{-4}$ to $2 \cdot 10^{-3}$: within 0.001.
- 12 modes chosen on validation: 32 modes cost 7× the parameters and run 3.6× slower for no gain.

## 8 Why the MLP cannot catch up




10× more parameters, 3× more steps or 10× more data leave the MLP near 0.035 to 0.046. An FNO trained on only **400** maps (0.026) beats an MLP trained on **9,700** (0.036). Likely reason (not tested): without spatial structure it must relearn the same local physics at every pixel.

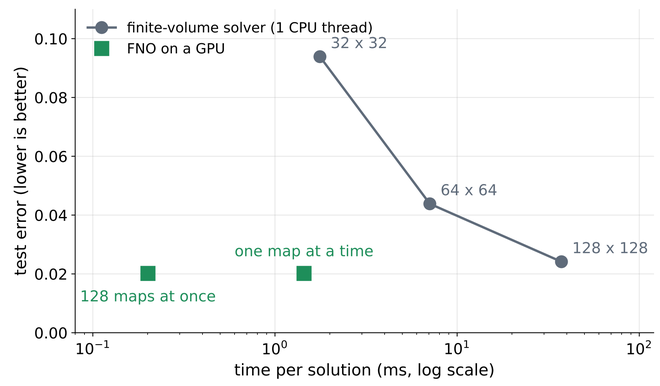
## 9 Data and resolution




- FNO error: 100 maps 0.044, 400 maps 0.026, 900 maps 0.022, 5,000 maps 0.020; levels off near 0.02.
- Same-grid error at $32^2/64^2/128^2$: 0.027 / 0.022 / 0.021. A $64^2$ model is 2 to 3× worse on another grid.

## 10 FNO versus a classical solver

Our cell-centred finite-volume solver: a water balance in every cell, face permeability by harmonic mean, one sparse system $Au = b$ solved by sparse LU, boundary treatment calibrated on validation data.



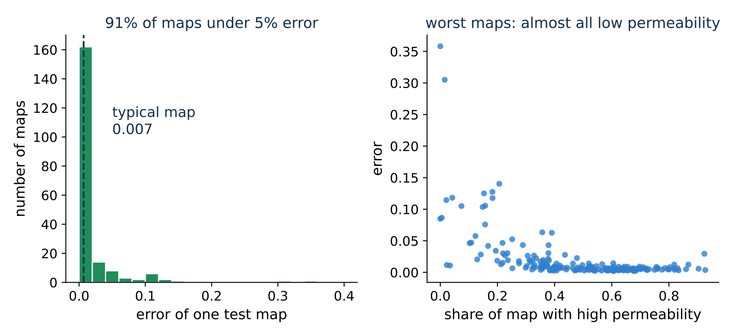

- At $64^2$: FNO error **0.020** vs solver 0.044; solver 7.1 ms per map (one CPU thread).
- FNO on a GPU: 1.4 ms per map, 0.2 ms per map in batches of 128, i.e. **5–35× faster**.
- On one CPU core the solver is faster (7 vs 37 ms); training pays off after about $5 \times 10^4$ solves.

## 11 Does it obey the physics?

**0.1157** — edge level (truth 0.1159)

**1.7 %** — error in mean pressure

**0.0015 %** — pixels below zero




91 % of test maps are under 5 % error. The worst maps are almost entirely low-permeability: tall pressure, rare in training (27 of 900), right shape but wrong height (up to 36 %).

## 12 Robustness (added after review)

**Noisy labels.** Gaussian noise (1, 5, 10 % of each map) on the training pressure maps, clean test maps:




At 10 % noise: FNO 0.025 → 0.036, MLP 0.045 → 0.051. The FNO stays ahead at every level.

**Self-adjusting sample weights.** Each map's weight follows its own running error. It finds the 27 hard training maps (weight ≈ 2.7×), but their test error does not improve (0.170 vs 0.172) and the overall error rises slightly (0.0249 → 0.0265): a negative result.

## 13 Conclusions and limitations

- **Training mattered most:** matching loss and metric, one learning-rate decay and symmetry augmentation halved the error.
- **Structure beats size:** the FNO is 2× more accurate than a tuned MLP with 10× fewer parameters, and robust to label noise.
- **Limits:** one PDE and dataset; agreement with the data, not the exact PDE; tied to the training grid; speed gain needs a GPU; nearly uniform maps remain hard.

Refs: Li et al., FNO, ICLR 2021; Takamoto et al., PDEBench, NeurIPS D&B 2022. 167 training runs; code, per-run results and notebook in the project repository. AI tools were used for coding and drafting; all numbers come from real runs.